BROS Technology

Project Handover Document

Electronics Marketplace Platform

Prepared by: Fira Tech Solutions

Date: August 2026

00 — System Overview

BROS Technology is an electronics device marketplace for selling iPhones, Samsungs, iPads, MacBooks, Laptops, AirPods, and Smartwatches in Ethiopia. Customers browse a public storefront, inquire about products via Telegram, WhatsApp, or phone call, and a small team of agents and administrators manage the catalog, syndicate products to a Telegram channel, and track assets and commissions — all through a mobile admin app and a web admin portal.

System Architecture

```
graph TB
    subgraph "Client Surfaces"
        PUBLIC["Public Website<br/>TanStack Start (SSR)<br/>bros-technology.vercel.app"]
        ADMIN_APP["Admin Mobile App<br/>React Native / Expo<br/>com.brostechnology.admin"]
        ADMIN_PORTAL["Admin Web Portal<br/>React + Vite<br/>bros-technology-admin.vercel.app"]
    end

    subgraph "Backend"
        API["Express.js API<br/>bros-technology-api.vercel.app"]
    end

    subgraph "Data & Services"
        DB[(PostgreSQL<br/>Supabase)]
        CLOUDINARY["Cloudinary<br/>Image Storage"]
        BREVO["Brevo<br/>Email Service"]
        TG["Telegram Bot API<br/>Channel Syndication"]
    end

    PUBLIC -->|HTTP/REST| API
    ADMIN_APP -->|HTTP/REST| API
    ADMIN_PORTAL -->|HTTP/REST| API
    API -->|Prisma ORM| DB
    API -->|Upload/Destroy| CLOUDINARY
    API -->|Send Email| BREVO
    API -->|Send Messages| TG
```

Deployed Surfaces

Surface	URL	Status
Backend API	`https://bros-technology-api.vercel.app`	Production
Public Website	`https://bros-technology.vercel.app`	Production
Admin Web Portal	`https://bros-technology-admin.vercel.app`	Production
Admin Mobile App	`com.brostechnology.admin` (APK / iOS)	Production

Tech Stack

Component	Framework / Language	Key Libraries	Hosting
Backend API	Node.js + Express 5 (ES modules)	Prisma 6.9, Sharp, Cloudinary, Brevo, JWT, Multer	Vercel (serverless)
Database	PostgreSQL 15	Prisma ORM	Supabase (EU-West-3)
Public Website	React 19 + TanStack Start (SSR)	TanStack Router, React Query, Framer Motion, Three.js, Tailwind CSS v4, shadcn/ui	Vercel (Nitro)
Admin Web Portal	React 19 + Vite 8	React Router v7, TanStack Query v5, Recharts, Tailwind CSS v4, Lucide React	Vercel (static SPA)
Admin Mobile App	React Native 0.85 + Expo SDK 56	React Navigation, Axios, Expo SecureStore, Expo Notifications, Reanimated	EAS Build (APK/iOS)
Image Processing	Sharp 0.33	Resize to 1200px, convert to WebP quality 80	—
Email	Brevo (Sendinblue)	Password reset emails with 6-digit code	—
Telegram	Bot API	Channel posting, inline keyboards, Mini App	—

Key Business Rules

- **Inquiry-based commerce**: No cart/checkout. Customers contact the shop directly via Telegram, WhatsApp, or phone call.
- **Dynamic product schema**: Each category defines its own attribute fields (e.g., iPhones need `brand`, `model`, `storage`, `carrier`; Laptops need `processor`, `gpu`, `screenSize`).
- **Agent code system**: New agents register using a 6-digit code generated by the admin. Agents can only see and manage their own listings.
- **Telegram syndication**: Products are posted to a Telegram channel with category-aware captions and inline keyboard buttons for location, browsing, and ordering.
- **Ethiopian market focus**: All prices in ETB (Ethiopian Birr). Product options pre-filled with brands/models popular in Ethiopia.

Default Credentials

Email	Password	Role
`admin@brostechnology.com`	`Admin@12345`	SUPER_ADMIN

> **Warning**: Change this password immediately after first deployment.

03 — Features

Admin Mobile App

Login (`src/screens/LoginScreen.js`)

- Email/password form with show/hide password toggle
- Brand showcase panel (hidden on mobile)
- Links to "Forgot password?" and "Register as Agent"
- Footer links to firatech.systems

Agent Signup (`src/screens/AgentSignupScreen.js`)

- **Step 1**: Enter 6-digit invitation code → `POST /api/auth/verify-agent-code`

- **Step 2**: Complete registration (name, email, phone, password) → `POST /api/auth/register`
- Code verification pre-fills name/phone from server

Dashboard (`src/screens/DashboardScreen.js`)

- StatusChart: bar/pie chart of listing statuses (Available/Sold/Pending/Archived)
- RecentProducts: first 8 listings with pull-to-refresh
- Notification bell with unread count badge

Properties (`src/screens/PropertiesScreen.js`)

- Two tabs: **Listings** and **Categories**
- Listings tab: search bar, category filter chips, paginated FlatList of ListingCards
- Inline Telegram syndication button per listing
- Categories tab: CRUD for categories with schema rules (dynamic field definitions)
- Floating "+" button for add listing

Add Listing (`src/screens/AddListingScreen.js`)

- **3-step wizard**:
- Step 1: Title, description, price, stock, category selector
- Step 2: Dynamic schema fields from category (brand-dependent model selection, predefined dropdowns with "Other" custom option)
- Step 3: Image picker with drag-to-reorder, listing summary
- Submits as FormData with images

Listing Detail (`src/screens/ListingDetailScreen.js`)

- Edit all fields: title, description, price, stock, status (Available/Pending/Sold/Archived)
- Dynamic schema fields (same system as Add Listing)
- Image management: existing images (add/remove), new uploads
- Save and Delete actions

Settings (`src/screens/SettingsScreen.js`)

- Profile card (tap to navigate to Profile)
- Appearance toggle (dark/light)
- Language dropdown (English/Amharic/Afaan Oromoo)
- Store section (admin-only): Contact & Social Media
- About section with version
- Logout button

Profile (`src/screens/ProfileScreen.js`)

- Avatar upload (image picker)
- Name, email fields
- Contact & Social Media: 10 preset fields (Phone, WhatsApp, Telegram, Facebook, Twitter, Instagram, TikTok, YouTube, LinkedIn, Website) + custom fields

Syndication Config (`src/screens/SyndicationConfigScreen.js`)

- **Settings tab** (admin-only): Bot token, channel ID, active toggle, test connection
- **Posts tab**: Swipeable filter pages (All/Successful/Pending/Failed)
- Post cards with image, title, status badge, action badge, timestamp
- Detail modal: caption editor, delete, retry (for failed)

Finance (`src/screens/CommissionScreen.js`)

- Summary cards: Total Assets, Total Value, Available, Sold, Pending, Archived
- Per-category breakdown with status pills and progress bars
- Admin-only

Notifications (`src/screens/NotificationScreen.js`)

- Full-screen notification list with type icons
- Tap marks as read and navigates to relevant listing
- "Mark all read" button
- Polls every 30 seconds via `useNotifications` hook

Contact Settings (`src/screens/ContactSettingsScreen.js`)

- Admin-only store settings editor
- Sections: Contact Information, Business Details, Social Media
- All fields stored via `PUT /api/settings`

Admin Web Portal

Dashboard (`src/pages/Dashboard.tsx`)

- Uses `useListings()` + `useCategories()` (TanStack Query)
- Stats: Total Products, Available, Sold, Total Stock, Categories
- Device Analytics by Category with icons
- Status Distribution horizontal bar chart (custom, not Recharts)
- Recent Products list (latest 5)

Properties (`src/pages/Properties.tsx`)

- DataTable with columns: Product (image+title+category), Price, Status, Stock (color-coded), Agent, Date, Actions
- Search by title/category, filter by category dropdown
- Actions: Edit, Syndicate, Delete (with confirmation modals)
- Pagination: 15 per page

Add Listing (`src/pages/AddListing.tsx`)

- 3-step wizard (same UX as mobile app)
- Dynamic schema fields with custom dropdowns (predefined options + "Other" custom)
- Brand-dependent model selection
- Submits as FormData

Listing Detail (`src/pages/ListingDetail.tsx`)

- Two-column layout (main + 320px sidebar)
- Basic info card, dynamic category fields, product photos sidebar
- Save Changes, Delete, Syndicate actions

Categories (`src/pages/Categories.tsx`)

- DataTable: Category (icon+name), Fields count, Products count, Actions
- Create/Edit modal with icon selector and Schema Rules editor
- Add rules: field name, type (string/number/boolean/select), required toggle

Agents (`src/pages/Agents.tsx`)

- Two tabs: Invitation Codes / Registered Agents
- Generate code modal: name, role toggle, max uses
- Copy-to-clipboard for codes
- Revoke/Remove with confirmation

Syndication (`src/pages/Syndication.tsx`)

- **Settings tab** (admin): Bot info, channel info, configuration modal
- **Posts tab**: Filter pills, post list with status/action badges
- Detail modal: caption editor, delete, retry

Finance (`src/pages/Finance.tsx`)

- Uses `useAssetStats()` (TanStack Query)

- Stats cards, Recharts BarChart by category, category detail cards

Settings (`src/pages/Settings.tsx`)

- Profile card, Appearance toggle, Language dropdown
- Store section (admin-only): Contact, Business, Social Media fields
- About, Logout

Public Website

Homepage (`src/routes/index.tsx`)

- Sticky parallax hero with responsive images (AVIF/WebP)
- 3D Three.js canvas (floating golden icosahedron)
- Categories grid (fetched from API)
- Featured products (first 6 from API)
- Animated stats counters (2500+ customers, 4800+ products, etc.)
- Brand marquee with motion animation
- SEO content section about BROS Technology
- Contact section with social links

Catalog (`src/routes/catalog.tsx`)

- Full-text search with debouncing (300ms), Cmd+K shortcut
- Desktop sidebar with `FilterPanel` (price range, category-specific filters)
- Mobile filter sheet (bottom drawer)
- Horizontal category pills with animated indicator
- Grid and list layout toggle
- Zod schema validation for search params
- `CollectionPageJsonLd` structured data

Product Detail (`src/routes/property.$id.tsx`)

- SSR loader for crawlers: `loader: async ({ params }) => fetchProduct(params.id)`
- Hero image with parallax effect
- Specs table, description, gallery with lightbox
- Lightbox: keyboard navigation, touch swipe, thumbnails
- Sticky order sidebar (desktop): Telegram, WhatsApp, Call buttons
- Mobile sticky order bar
- `ProductJsonLd` structured data
- Inquiry tracking: `POST /api/public/listings/:id/inquiry`

Sitemap (`src/routes/sitemap[.]xml.tsx`)

- Server-only dynamic XML sitemap
- Fetches all categories and products (up to 5000)
- Includes homepage, catalog, category pages, individual product URLs
- Cache: `s-maxage=3600, stale-while-revalidate=86400`

Telegram Mini App Integration

- Detects Telegram WebApp environment
- Syncs theme (dark/light) from Telegram
- Hides nav bar in MiniApp mode
- Uses Telegram `MainButton` for ordering
- `startParam` for deep linking to specific products

05 — Deployment

Backend (Vercel)

Automatic Deployment

The backend is deployed to Vercel via the `backend/` directory. Vercel detects the `vercel.json` and builds accordingly:

```
{
  "version": 2,
  "builds": [{ "src": "api/index.js", "use": "@vercel/node" }],
  "routes": [{ "src": "/(.*)", "dest": "api/index.js" }]
}
```

Build command (defined in `package.json`):

```
"vercel-build": "npx prisma generate"
```

Deployment Steps

1. Push to the `master` branch of the Git repository
2. Vercel automatically triggers a build
3. `prisma generate` runs during build to create the Prisma Client

4. The Express app is deployed as a Vercel serverless function

Post-Deploy Manual Steps

1. **Run database migrations** (if schema changed): ```bash # Via Vercel CLI vercel env pull .env.local npx prisma migrate deploy

Or via Supabase SQL Editor # Paste the migration SQL and execute ```

1. **Seed categories** (if new default categories added): `GET https://bros-technology-api.vercel.app/api/admin/seed-categories` (Requires admin authentication)

2. **Verify health**: `GET https://bros-technology-api.vercel.app/health`

Environment Variables in Vercel Dashboard

Set these in the Vercel project settings → Environment Variables:

Variable	Value
`DATABASE_URL`	Supabase pooled connection string
`DIRECT_URL`	Supabase direct connection string
`JWT_SECRET`	Strong random string (32+ chars)
`STORAGE_PROVIDER`	`cloudinary`
`CLOUDINARY_CLOUD_NAME`	Your Cloudinary cloud name
`CLOUDINARY_API_KEY`	Your Cloudinary API key
`CLOUDINARY_API_SECRET`	Your Cloudinary API secret
`CLOUDINARY_FOLDER`	`Home/listings`
`BREVO_API_KEY`	Your Brevo API key
`BREVO_FROM_EMAIL`	`girmasamuel200@gmail.com`
`NODE_ENV`	`production`

> **Important**: The `ALLOWED_ORIGINS` env var in Vercel should include the public website URL if it's not already hardcoded in `app.js`. The production origins are hardcoded in

> `app.js` : - `https://bros-technology-admin.vercel.app` - `https://bros-technology.vercel.app`

Public Website (Vercel)

Deployment Steps

1. Push to the `master` branch
2. Vercel detects the project and runs: `bash vite build`
3. Output is deployed as a Vercel Edge/Serverless function (Nitro preset)

Environment Variables in Vercel Dashboard

Variable	Value
`VITE_API_URL`	`https://bros-technology-api.vercel.app`
`VITE_SITE_URL`	`https://bros-technology.vercel.app`
`VITE_TELEGRAM_BOT_USERNAME`	`brostechnology`

Admin Web Portal (Vercel)

Deployment Steps

1. Push to the `master` branch
2. Vercel runs: `bash tsc -b && vite build`
3. Output is a static SPA deployed to Vercel

Vercel Configuration

```
{
  "rewrites": [{ "source": "/((?!assets/).*)", "destination": "/index.html" }],
  "redirects": [{ "source": "/favicon.ico", "destination": "/favicon-96x96.png", "permanent": fa
}
```

The SPA rewrite ensures client-side routing works (all paths serve `index.html`).

Environment Variables

None required. The API URL is hardcoded in `src/lib/api.ts`.

Admin Mobile App (EAS Build)

Building APK (Android)

```
# Install EAS CLI
npm install -g eas-cli

# Login to Expo
eas login

# Build APK (preview profile produces APK, not AAB)
eas build --platform android --profile preview

# Or for production
eas build --platform android --profile production
```

Build Profiles (`eas.json`)

Profile	Output	Distribution
`development`	Dev client	Internal
`preview`	APK	Internal
`production`	APK (Android), auto-increment (iOS)	Store

App Configuration (`app.config.js`)

Field	Value
Name	BROS Technology
Slug	`admin-app`
Version	1.1.0
Bundle ID	`com.brostechnology.admin`
EAS Project ID	`d706b125-4daf-4c2c-860a-537b89af730a`

Field	Value
Owner	`codearchitect001`

Submitting to Stores

```
# Android (Google Play)
eas submit --platform android

# iOS (App Store)
eas submit --platform ios
```

Database Migrations

How Migrations Are Handled

The project uses **Prisma Migrate** (not `prisma db push` for production).

```
# Create a new migration (development)
npm run prisma:migrate

# Apply migrations in production
npx prisma migrate deploy
```

Migration History

There are 14 migrations in `prisma/migrations/`:

1. `20260601175413_init` — Initial schema
2. `20260611210715_add_syndication_config` — SyndicationConfig table
3. `20260612061032_add_syndication_action_messageid` — SyndicationAction enum + columns
4. `20260613040002_add_profile_image_to_user` — User profileImage
5. `20260613043309_add_social_media_fields` — 9 social media columns
6. `20260613061124_add_telegram_chat_id` — Telegram chat fields (later removed from schema)
7. `20260613070737_add_telegram_user_info` — Telegram user fields (later removed from schema)
8. `20260617000001_add_custom_socials` — User customSocials JSON
9. `20260617000002_add_notifications` — Notifications table

10. `20260619054358_add_password_reset_fields` — Password reset token/expires
11. `20260619120000_add_commission_percent` — Listing commissionPercent
12. `20260808000001_make_agentid_nullable` — agentId nullable + SetNull
13. `20260809000001_add_stock_quantity` — Listing stockQuantity
14. `20260809000002_remove_city_neighborhood` — Drop city/neighborhood columns

Applying Migrations on Supabase

Option 1: Via Vercel CLI

```
vercel env pull .env.local
npx prisma migrate deploy
```

Option 2: Via Supabase SQL Editor - Open Supabase dashboard → SQL Editor - Paste the migration SQL from `prisma/migrations/YYYYMMDDHHMMSS_name/migration.sql` - Execute

Option 3: Via Prisma (local)

```
npx prisma migrate deploy
```

Rollback Procedure

If a Backend Deploy Breaks Something

1. **Vercel rollback**: Go to Vercel dashboard → Deployments → find the last working deployment → "Promote to Production"

2. **Database rollback**: If a migration was applied that broke things: ```bash # Create a reversal migration npx prisma migrate dev --create-only --name rollback_description

Edit the generated SQL to reverse the changes # Apply it npx prisma migrate deploy ```

1. **Emergency**: Revert the Git commit and push: `bash git revert HEAD git push`

If a Website Deploy Breaks Something

1. **Vercel rollback**: Same as backend — promote last working deployment

If the Mobile App Has a Critical Bug

1. **Fix and rebuild**: Push a fix, rebuild APK, distribute via internal testing

2. **Users on old version**: The app calls the backend API, so as long as the API is compatible, old versions continue to work. Breaking API changes require app updates.

06 — Brand & Design System

Color Palette

Primary Colors

Token	Hex	Usage
Primary Blue	`#1878B4`	Buttons, links, active states, brand accent
Primary Dark	`#125E8C`	Hover states, darker variant
Primary Tint	`#EAF4FB`	Light backgrounds, tag fills
Primary Hover	`#15699E`	Button hover
Accent Black	`#0A0A0A`	High-contrast accent

Background Colors

Token	Light Mode	Dark Mode	Usage
Background	`#F7F9FB`	`#0F1117`	Page background
Surface	`#FFFFFF`	`#1A1D27`	Card/panel background
Border	`#E7ECF1`	`#2A2D3A`	Dividers, card borders

Text Colors

Token	Light Mode	Dark Mode	Usage
Text	`#14181C`	`#E8EAED`	Primary text
Text Secondary	`#374151`	`#D1D5DB`	Secondary text
Text Muted	`#6B7280`	`#9CA3AF`	Labels, timestamps

Status Colors

Status	Color	Tint (Light)	Usage
Success / Available	`#22C55E`	`#DCFCE7`	In stock, successful syndication
Warning / Pending	`#F59E0B`	`#FEF3C7`	Pending syndication, low stock
Danger / Sold / Failed	`#EF4444`	`#FEE2E2`	Sold out, failed syndication, delete actions
Archived / Inactive	`#6B7280`	—	Archived listings, inactive states
Active / Primary	`#1878B4`	—	Active syndication status

Dark Mode

Dark mode is toggled via a `.dark` class on the `<html>` element. All CSS custom properties are overridden in the `.dark` selector. Theme preference is persisted to `localStorage`.

Admin Portal (`src/index.css`): Full dark mode token set in `.dark` class.

Admin App (`src/config/theme.js`): Complete light/dark theme object with `colors`, `spacing`, `radii`, `shadows`, `typography`.

Public Website (`src/styles.css`): OKLCH color system with dark mode via `html.light` class (inverted naming — dark is default).

Typography

Font Families

Context	Font	Weight	Source
Headings	Poppins	600, 700	Google Fonts (admin-portal `index.html`)
Body	Inter	400, 500, 600	Google Fonts (admin-portal `index.html`)
Ethiopic	Noto Sans Ethiopic	400	Google Fonts (public website `__root.tsx`)
Serif	Instrument Serif	—	Google Fonts (public website `__root.tsx`)

Type Scale (Admin Portal)

Element	Size	Weight	Font
Page title	24px	700	Poppins
Section heading	16px	600	Poppins
Body text	14px	400–500	Inter
Small text / labels	13px	600	Inter
Caption / timestamp	12px	400	Inter
Badge text	11px	600	Inter
Stat value	32px	700	Poppins

Spacing Scale

Token	Value
`--space-1`	4px
`--space-2`	8px
`--space-3`	12px
`--space-4`	16px
`--space-5`	24px
`--space-6`	32px
`--space-7`	48px
`--space-8`	64px

Border Radius

Token	Value	Usage
`--radius-sm`	8px	Small buttons, badges
`--radius-md`	12px	Inputs, cards

Token	Value	Usage
`--radius-lg`	16px	Modals, large cards
`--radius-xl`	24px	Special elements

Shadows

Token	Value
`--shadow-sm`	`0 1px 2px rgba(10,10,10,0.06)`
`--shadow-md`	`0 4px 12px rgba(10,10,10,0.08)`
`--shadow-lg`	`0 12px 32px rgba(10,10,10,0.12)`

Transitions

Token	Value
`--transition-fast`	150ms ease
`--transition-normal`	200ms ease

Brand Assets

Logo & Icon Files

File	Location	Used By
`bros_icon_concept4_monogram_clean.png`	`admin-app/assets/`	App icon, adaptive icon, splash
`bros_splash_full_concept4.png`	`admin-app/assets/`	Splash screen
`device-illustration.png`	`admin-app/assets/`	Login screen hero
`favicon-96x96.png`	`admin-portal/public/`, `public-website/public/images/favicon/`	Browser tab icon

File	Location	Used By
`hero.png`	`admin-portal/src/assets/`	Admin portal hero
`bros_desktop_light_HD.jpg`	`public-website/public/images/`	Desktop hero (light)
`bros_desktop_dark_HD.jpg`	`public-website/public/images/`	Desktop hero (dark)
`bros_mobile_light_HD.png`	`public-website/public/images/`	Mobile hero (light)
`bros_mobile_dark_HD.png`	`public-website/public/images/`	Mobile hero (dark)
Brand logos	`public-website/public/images/brands/`	Brand marquee on homepage

Splash Screen

- Background: `#1878B4` (primary blue)
- Image: `bros_splash_full_concept4.png`
- Resize mode: `contain`

App Icons

- iOS: `bros_icon_concept4_monogram_clean.png` (1024x1024)
- Android adaptive: Same image with `#1878B4` background

Component Patterns

Button Variants

Variant	Background	Text	Hover
`primary`	`#1878B4`	White	`#125E8C`
`secondary`	Transparent	`#1878B4`	`#EAF4FB`
`ghost`	Transparent	`#6B7280`	`#F7F9FB`
`danger`	`#EF4444`	White	`#DC2626`

Variant	Background	Text	Hover
`danger-tint`	`#FEE2E2`	`#EF4444`	`#FECACA`
`success`	`#22C55E`	White	`#16A34A`

Input Fields

- Height: 44px
- Border: 1px solid `var(--color-border)`
- Border radius: `var(--radius-md)` (12px)
- Focus ring: `0 0 0 3px rgba(24,120,180,0.15)`
- Error state: Red border + error message text

Status Badges

- Pill shape with colored dot (6px circle)
- Two sizes: `sm` (padding 3px/10px, font 12px) and `md` (padding 5px/14px, font 13px)
- Label auto-formatted: first char uppercase, rest lowercase

Data Tables

- Row hover highlighting
- Pagination: page number buttons (max 5 visible, sliding window)
- Shows "Showing X-Y of Z" counter
- Action buttons visible on hover (desktop), always visible (mobile)

Tech Debt / Inconsistencies

- **ContactSettings.tsx** uses Tailwind utility classes (`className="..."`) instead of the inline styles + CSS custom properties pattern used by all other admin-portal pages. This is inconsistent with the design system approach.
- **Public Website** uses OKLCH color space while admin-portal/app use hex colors. The color values are semantically equivalent but technically different.
- **Public Website** dark mode uses `html.light` class (inverted naming — dark is default), while admin-portal uses `html.dark` class (dark mode added via `.dark` class). This inconsistency could confuse developers switching between codebases.
- **Admin App** has a separate `theme.js` file with color tokens that should be kept in sync with the admin-portal's `index.css` tokens manually. No shared token source.